



langgraph-swarm

v0.1.0

license

MIT

downloads

501k

See the [project description](#) for more details.

mod swarm

FUNCTION	DESCRIPTION
<code>create_swarm</code>	Create a multi-agent swarm.
<code>add_active_agent_router</code>	Add a router to the currently active agent to the <code>StateGraph</code> .

class SwarmState

Bases: `MessagesState`

State schema for the multi-agent swarm.

func create_swarm

```
create_swarm(  
    agents: list[Pregel],  
    *,  
    default_active_agent: str,  
    state_schema: StateSchemaType = SwarmState,  
    context_schema: type[Any] | None = None,  
    **deprecated_kwargs: Unpack[DeprecatedKwargs],  
) -> StateGraph
```

Create a multi-agent swarm.

PARAMETER	DESCRIPTION
<code>agents</code>	List of agents to add to the swarm An agent can be a LangGraph <code>CompiledStateGraph</code>, a functional API <code>workflow</code>, or any other <code>Pregel</code> object. TYPE: <code>list[Pregel]</code>
<code>default_active_agent</code>	Name of the agent to route to by default (if no agents are currently active). TYPE: <code>str</code>
<code>state_schema</code>	State schema to use for the multi-agent graph. TYPE: <code>StateSchemaType</code> DEFAULT: <code>SwarmState</code>
<code>context_schema</code>	Specifies the schema for the context object that will be passed to the workflow. TYPE: <code>type[Any] None</code> DEFAULT: <code>None</code>

RETURNS	DESCRIPTION
<code>StateGraph</code>	A multi-agent swarm <code>StateGraph</code> .

 Example

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from langgraph_swarm import create_handoff_tool, create_swarm

def add(a: int, b: int) -> int:
    '''Add two numbers'''
    return a + b

alice = create_agent(
    "openai:gpt-4o",
    tools=[
        add,
        create_handoff_tool(
            agent_name="Bob",
            description="Transfer to Bob",
        ),
    ],
    system_prompt="You are Alice, an addition expert.",
    name="Alice",
)

bob = create_agent(
    "openai:gpt-4o",
    tools=[
        create_handoff_tool(
            agent_name="Alice",
            description="Transfer to Alice, she can help with math",
        ),
    ],
    system_prompt="You are Bob, you speak like a pirate.",
    name="Bob",
)

checkpointer = InMemorySaver()
workflow = create_swarm(
    [alice, bob],
    default_active_agent="Alice"
)
app = workflow.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "1"}}
turn_1 = app.invoke(
    {"messages": [{"role": "user", "content": "i'd like to speak to Bob"}]},
    config,
)

turn_2 = app.invoke(
    {"messages": [{"role": "user", "content": "what's 5 + 7?"}]},
    config,
)
```

`func` `add_active_agent_router`

```
add_active_agent_router(  
    builder: StateGraph, *, route_to: list[str], default_active_agent: str  
) -> StateGraph
```

Add a router to the currently active agent to the `StateGraph` .

PARAMETER	DESCRIPTION
<code>builder</code>	The graph builder (<code>StateGraph</code>) to add the router to. TYPE: <code>StateGraph</code>
<code>route_to</code>	A list of agent (node) names to route to. TYPE: <code>list[str]</code>
<code>default_active_agent</code>	Name of the agent to route to by default (if no agents are currently active). TYPE: <code>str</code>

RETURNS	DESCRIPTION
<code>StateGraph</code>	<code>StateGraph</code> with the router added.

 Example

```

from langchain.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from langgraph.graph import StateGraph
from langgraph_swarm import SwarmState, create_handoff_tool,
add_active_agent_router

def add(a: int, b: int) -> int:
    '''Add two numbers'''
    return a + b

alice = create_agent(
    "openai:gpt-4o",
    tools=[
        add,
        create_handoff_tool(
            agent_name="Bob",
            description="Transfer to Bob",
        ),
    ],
    system_prompt="You are Alice, an addition expert.",
    name="Alice",
)

bob = create_agent(
    "openai:gpt-4o",
    tools=[
        create_handoff_tool(
            agent_name="Alice",
            description="Transfer to Alice, she can help with math",
        ),
    ],
    system_prompt="You are Bob, you speak like a pirate.",
    name="Bob",
)

checkpointer = InMemorySaver()
workflow = (
    StateGraph(SwarmState)
    .add_node(alice, destinations=("Bob",))
    .add_node(bob, destinations=("Alice",))
)
# this is the router that enables us to keep track of the last active agent
workflow = add_active_agent_router(
    builder=workflow,
    route_to=["Alice", "Bob"],
    default_active_agent="Alice",
)

# compile the workflow
app = workflow.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "1"}}
turn_1 = app.invoke(

```

```
    {"messages": [{"role": "user", "content": "i'd like to speak to Bob"}]},  
    config,  
)  
turn_2 = app.invoke(  
    {"messages": [{"role": "user", "content": "what's 5 + 7?"}]},  
    config,  
)
```

mod handoff

FUNCTION	DESCRIPTION
<code>create_handoff_tool</code>	Create a tool that can handoff control to the requested agent.

func create_handoff_tool

```
create_handoff_tool(  
    *, agent_name: str, name: str | None = None, description: str | None = None  
) -> BaseTool
```

Create a tool that can handoff control to the requested agent.

PARAMETER	DESCRIPTION
<code>agent_name</code>	The name of the agent to handoff control to, i.e. the name of the agent node in the multi-agent graph. Agent names should be simple, clear and unique, preferably in <code>snake_case</code>, although you are only limited to the names accepted by LangGraph nodes as well as the tool names accepted by LLM providers (the tool name will look like this: <code>transfer_to_<agent_name></code>). TYPE: <code>str</code>
<code>name</code>	Optional name of the tool to use for the handoff. If not provided, the tool name will be <code>transfer_to_<agent_name></code>. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>

PARAMETER	DESCRIPTION
<code>description</code>	Optional description for the handoff tool. If not provided, the tool description will be <code>Ask agent <agent_name> for help</code>. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>